Modul 216 – Internet of Everything (IoE)

LB-Projekt «Letterbox»

Projektdokumentation

Auftraggeber: Noser Young AG, Worblaufen – Standort Lindenpark

Team: Yurii Myronenko, Cristiano Grgic, Noé Bouvard

Git-Repository: https://github.com/Noe-NoserYoung/uek216.git

Datum / Version: 06.11.2026 / 2.1

Inhaltsverzeichnis

1 Projektübersicht 4
1.1 Arbeitsaufteilung 4
2 Technische Anforderungen 5
2.1 Ausgangslage und Ziele 5
2.2 Eingesetzte Komponenten 5
2.3 Zusammenspiel der Komponenten 6
2.4 Wahl der Komponenten und Speisung 6
2.5 Umsetzung der Anforderungen 7
3 Praktische Anbindung der Geräte ans Netzwerk 8
3.1 Netzwerkplan 8
3.2 Komponenten im Netzwerk 9
3.3 Datenfluss und MQTT-Kommunikation 9
3.4 Schritte zur Internetverbindung der IoE-Geräte 10
3.5 Verbindungsverlust / Reconnect 10
4 Parameter und Konfigurationen 11
4.1 Übersicht der Parameter 11
4.2 Automatisierung 1: Posterkennung 11
4.3 Automatisierung 2: Alarm 14
4.4 Abgleich mit den Anforderungen 18
5 Einbindung der Geräte 19
5.1 Verdrahtung 19
5.2 Fotos der Geräte 20
5.3 Erfahrungen beim Aufbau 21
6 Erweiterungen und Weiterführung 23
6.1 Mögliche Erweiterung: weitere Firmen der Noser Gruppe 23
6.2 Codequalität und Coding Conventions 23
6.3 Unterlagen zum Fortführen der Arbeit 24
7 Zugriffskonzept für Geräte und Daten 26
8 Schutzkonzept für die Geräte 27
8.1 Ausgangslage / Risiken 27
8.2 Massnahme 1: Offline-Erkennung 27
8.3 Massnahme 2: Prüfung der Nachrichten 29
8.4 Übersicht 32
9 Konzept für regelmässige Updates 33
9.1 Vorgehen beim Aktualisieren 33
9.2 Quellen der Updates 33
9.3 OTA-Update (Over the Air) 33

10 Testfälle 35
10.1 Testprotokoll 35
Quellenverzeichnis 37

1 Projektübersicht

Autor/in: alle

1.1 Arbeitsaufteilung

Kapitel	Inhalt	Autor/in
1 Projektübersicht	Arbeitsaufteilung	alle
2 Technische Anforderungen	Ausgangslage, Komponenten	Cristiano
3 Anbindung ans Netzwerk	Netzwerkplan, Internetverbindung	Yurii
4 Parameter und Konfigurationen	Zwei Automatisierungen	Yurii
5 Einbindung der Geräte	Verdrahtung, Fotos, Aufbau	Cristiano
6 Erweiterungen	Erweiterung, Coding Conventions	Yurii
7 Zugriffskonzept	Wer darf was	Yurii
8 Schutzkonzept	Schutzmassnahmen	Yurii
9 Update-Konzept	OTA-Update	Noé
10 Testfälle	Testprotokoll	Cristiano
Code ESP32 Briefkasten		Yurii / Noé
Code ESP32 Alarm / Node-RED		Yurii / Noé

Tabelle 1: Arbeitsaufteilung

2 Technische Anforderungen

Autor/in: Cristiano Grgic

2.1 Ausgangslage und Ziele

Im Lindenpark haben mehrere Firmen der Noser Gruppe ihre Briefkästen. Diese werden nicht immer regelmässig geleert, und manchmal bleibt die Post mehrere Tage liegen. Deshalb soll ein intelligenter Briefkasten erkennen, ob Post da ist, den Status per MQTT senden und jeden Tag melden, wenn der Briefkasten nicht geleert wurde.

Unser Ziel war es, ein funktionelles Post-Erkennungssystem zu bauen. Die Probleme, die vorkamen, waren grob gesagt: schlechter Code und schlechte Verkabelungen.

Anforderung aus dem Auftrag	Art
Briefkasten der Noser Young AG mit geeigneten Sensoren überprüfen	Muss
Merken, ob sich Post im Briefkasten befindet	Muss
Status regelmässig an das MQTT-Topic «[standort]/letterbox/[firma]» senden	Muss
Mindestens täglich melden, wenn der Briefkasten nicht geleert wurde	Muss
Sensorwahl, Speisung und Umwelteinflüsse berücksichtigen	Muss
Alle Projektunterlagen im Git-Repository, für den Berufsbildner zugänglich	Muss
Weitere Firmen der Noser Gruppe	Optional
Status sichtbar machen (Node-RED, Dashboard, Lampe)	Optional

Tabelle 2: Anforderungen aus dem Projektauftrag

2.2 Eingesetzte Komponenten

Komponente	Gerät	Art	Anschluss	Aufgabe im Projekt
ESP32-WROOM	Briefkasten	Mikrocontroller	USB	Misst alle 2 s die Distanz und sendet den Status per MQTT
HC-SR04 (Ultraschall)	Briefkasten	Sensor	Trig GPIO 5, Echo GPIO 18	Misst den Abstand am Boden des Briefkastens (leer ca. 36 cm, Post: unter 5 cm oder verdeckt)
Powerbank	Briefkasten	Speisung	USB	Stromversorgung im Briefkasten ohne Steckdose
ESP32-WROOM	Alarm	Mikrocontroller	USB	Empfängt Status und Alarm per MQTT, steuert Lampe und Sirene

Komponente	Gerät	Art	Anschluss	Aufgabe im Projekt
LED-Modul (Lampe)	Alarm	Aktor	GPIO 2	Leuchtet, solange Post im Briefkasten liegt
Sirene / Buzzer	Alarm	Aktor	GPIO 4	Piept bei Alarm 30 s lang (Takt 500 ms)
Taster	Alarm	Sensor (Eingabe)	GPIO 13, GND	Schaltet die Sirene sofort aus
Breadboard, Jumper-Kabel	beide	Sonstiges	–	Verbindungen ohne Löten

Tabelle 3: Komponenten der IoE-Endgeräte

2.3 Zusammenspiel der Komponenten

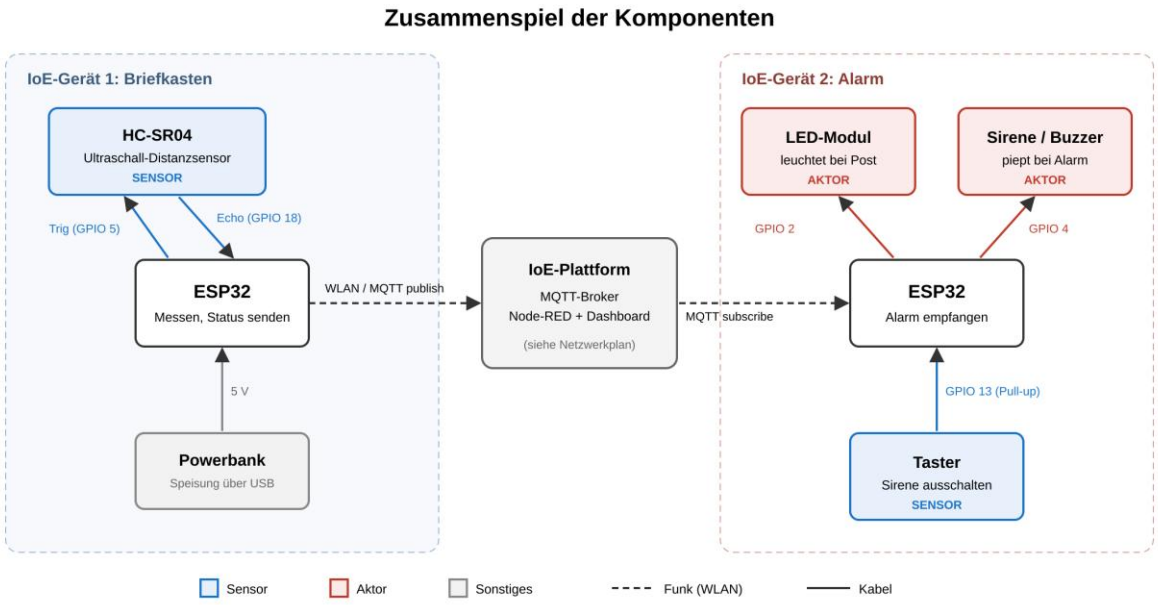


Abbildung 1: Zusammenspiel der Komponenten beider IoE-Endgeräte

2.4 Wahl der Komponenten und Speisung

Zuerst haben wir uns überlegt, welche Komponenten wir benötigen. Wir haben uns für einen ESP32-Chip, einen Ultraschallsensor (HC-SR04), eine Steckplatine und die passenden Kabel entschieden.

Anfangs dachten wir, dass ein Laser besser geeignet wäre. Allerdings wäre diese Lösung deutlich aufwendiger gewesen und hätte mehr Platz benötigt. Man braucht nämlich nicht nur einen Laser, sondern auch eine zuverlässige Oberfläche, die das Licht zurückreflektiert.

Danach haben wir die Komponenten provisorisch verkabelt. Unser Ziel war zunächst nicht eine perfekte Verkabelung, sondern zu testen, ob alles grundsätzlich funktioniert. Die Informationen zur Verkabelung haben wir mithilfe von KI erhalten.

Wir haben auch noch einen Alarm hinzugefügt, der warnt, wenn man vergisst, die Post zu nehmen.

Speisung

- **ESP32 Briefkasten:** Powerbank über USB, weil im Briefkasten keine Steckdose ist.
- **ESP32 Alarm:** mit einem USB-Kabel (USB-A auf Micro-USB) am Laptop angeschlossen, weil das Gerät drinnen neben dem Laptop steht.
- **Umwelteinflüsse:** Es gibt nur sehr wenige Einflüsse, ausser in sehr seltenen Extremfällen.

2.5 Umsetzung der Anforderungen

Wie die Anforderungen umgesetzt wurden, steht im Abgleich in Kapitel 4.4 (siehe Tabelle 7).

3 Praktische Anbindung der Geräte ans Netzwerk

Autor/in: Yurii Myronenko

In diesem Teil sehen Sie den Netzwerkplan als Bild, wie alles korrekt verbunden ist und welche Schritte nötig sind, damit die IoE-Endgeräte eine Internetverbindung erhalten.

3.1 Netzwerkplan

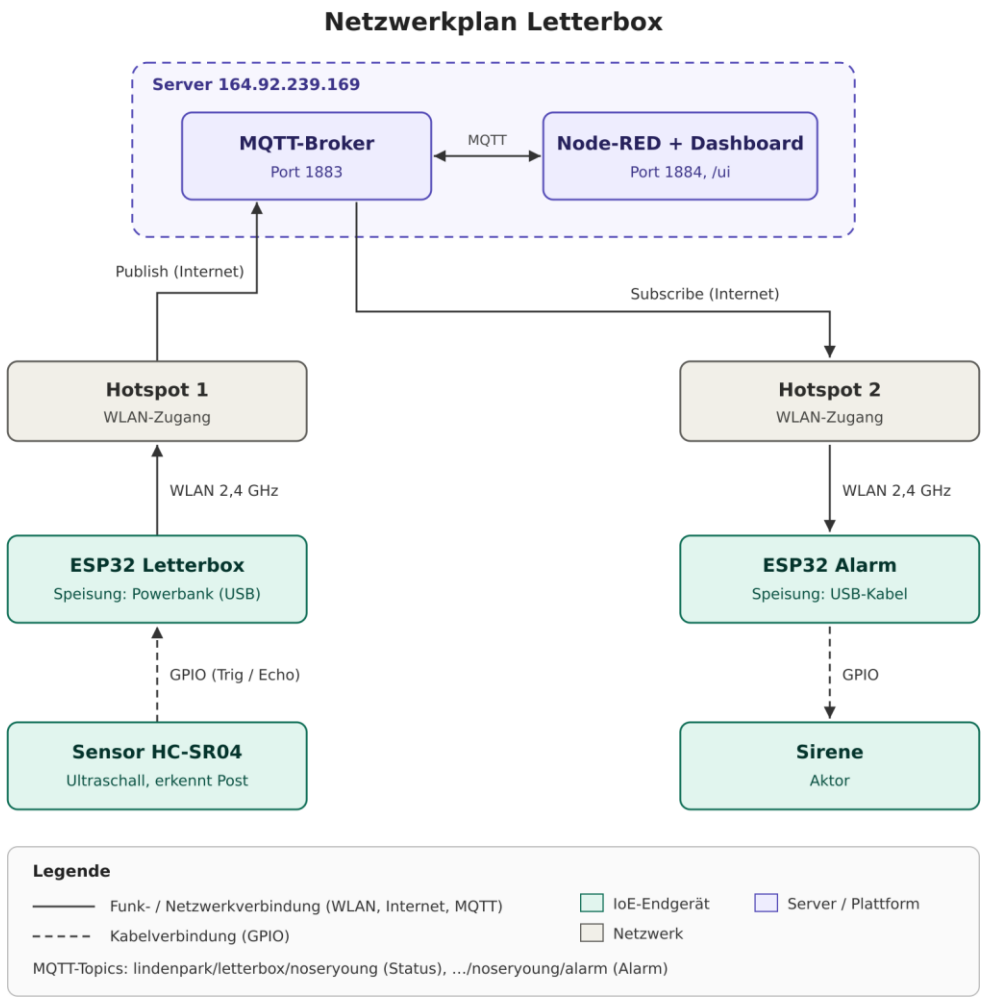


Abbildung 2: Netzwerkplan Letterbox

1. Wir verbinden den ESP32 mit dem Distanzsensor. Er schickt die Daten über den Hotspot an das Topic `lindenpark/letterbox/noseryoung`.
2. Der Broker nimmt diese Daten und schickt sie danach an Node-RED.
3. In Node-RED passiert die ganze Berechnung: ob ein Brief da ist oder nicht, wie spät es ist und so weiter.
4. Danach gehen alle diese Ergebnisse zum Broker. Der ESP32 mit dem Alarm hat das Topic abonniert (Subscribe) und prüft, ob ein Brief da ist und ob es schon zu spät ist. Dann schaltet er die Sirene ein.

Jeder ESP32 hat einen eigenen Hotspot, weil wir die ESP32 wegen technischer Probleme, die wir nicht selbst lösen können, nicht mit dem Firmen-WLAN verbinden können. Aus diesem Grund haben alle ESP32 eine Internetverbindung, auch wenn z. B. der Alarm oben ist und der andere ESP32 draussen kontrolliert, ob Post da ist. So können sie in jedem Fall kommunizieren und alles wird berechnet.

3.2 Komponenten im Netzwerk

Komponente	Rolle / Aufgabe
ESP32 Letterbox	Publisher
ESP32 Alarm	Subscriber
Hotspot 1 / 2	Internetverbindung
MQTT-Broker (164.92.239.169, Port 1883)	MQTT-Verbindung und Verteilung zwischen allen ESP32
Node-RED (http://164.92.239.169:1884)	Hauptlogik und Timer
Dashboard (http://164.92.239.169:1884/ui)	Zeigt den aktuellen Zustand

Tabelle 4: Komponenten im Netzwerk

3.3 Datenfluss und MQTT-Kommunikation

Was ist Publish? Publish bedeutet, dass ein Gerät (meistens ein Sensor) Daten teilt und an den Broker schickt.

Was ist Subscribe? Bei Subscribe erhält ein Gerät (meistens ein Aktor) Daten vom Broker.

Wir benutzen diese Topics:

- `lindenpark/letterbox/noseryoung`
- `lindenpark/letterbox/noseryoung/alarm`

Im Payload bekommen wir den Status, ob der Briefkasten leer oder voll ist ("mail":true oder "mail":false).

Der Sensor misst alle 2 Sekunden. Der ESP32 sendet drei Arten von Nachrichten:

Situation	Nachricht	Wann wird gesendet?
Der Status hat sich geändert (Post ist gekommen oder wurde abgeholt)	`"change"`	Direkt nach der Bestätigung, ca. 6 Sekunden nach dem Wechsel
Heartbeat („ich lebe noch“), auch ohne Änderung	`"heartbeat"`	Alle 5 Minuten
Nach der Neuverbindung mit MQTT	`"reconnect"`	Nach einem Verbindungsabbruch

Tabelle 5: Nachrichtentypen des ESP32 Letterbox

3.4 Schritte zur Internetverbindung der IoE-Geräte

1. Im Code schreiben wir den Namen (SSID) und das Passwort für den Hotspot und den Code für die Verbindung.

```
// ---------- WLAN ----------
void setup_wifi() {
  WiFi.mode(WIFI_STA);
  WiFi.begin(WIFI_SSID, WIFI_PASS);
  Serial.print("WLAN verbinden");
  while (WiFi.status() != WL_CONNECTED) {
    delay(500);
    Serial.print(".");
  }
  WiFi.setSleep(false);
  Serial.println();
  Serial.print("IP: ");
  Serial.println(WiFi.localIP());
}
```

Abbildung 3: Funktion setup_wifi() im Code

2. Dann verbindet sich der ESP32 mit dem WLAN und danach über die IP-Adresse und den Port mit dem Broker.

```
09:33:16.014 -> ets Jun  8 2016 00:22:57
09:33:19.089 ->
09:33:19.089 -> rst:0x1 (POWERON_RESET),boot:0x13 (SPI_FAST_FLASH_BOOT)
09:33:19.089 -> configsip: 0, SPIWP:0xee
09:33:19.089 -> clk_drv:0x00,q_drv:0x00,d_drv:0x00,cs0_drv:0x00,hd_drv:0x00,wp_drv:0x00
09:33:19.089 -> mode:DIO, clock div:1
09:33:19.089 -> load:0x3fff0030,len:4876
09:33:19.089 -> ho 0 tail 12 room 4
09:33:19.089 -> load:0x40078000,len:16532
09:33:19.089 -> load:0x40080400,len:3500
09:33:19.089 -> entry 0x400805b4
09:33:19.490 -> WLAN verbinden...
09:33:20.965 -> IP: 10.146.213.179
09:33:21.006 -> Bereit
09:33:21.209 -> MQTT verbunden
```

Abbildung 4: Ausgabe im Serial Monitor (WLAN und MQTT verbunden)

3.5 Verbindungsverlust / Reconnect

Wenn ein ESP32 die Verbindung zum WLAN verliert, verbindet er sich automatisch neu. Dafür prüft er am Anfang von `loop()`, ob das WLAN noch verbunden ist. Wenn nicht, ruft er `setup_wifi()` auf (siehe Abbildung 11). Danach verbindet er sich auch wieder mit dem MQTT-Broker und sendet die Nachricht `"reconnect"`. So muss man den ESP32 nicht mehr manuell mit der EN-Taste neu starten.

4 Parameter und Konfigurationen

Autor/in: Yurii Myronenko

Unser System hat zwei Automatisierungen. Die erste erkennt, ob Post im Briefkasten liegt. Die zweite prüft jeden Tag, ob der Briefkasten geleert wurde, und löst sonst einen Alarm aus. In diesem Kapitel zeigen wir die wichtigsten Einstellungen und erklären, warum wir sie so gewählt haben.

4.1 Übersicht der Parameter

Parameter	Wert	Wo?	Warum?
WLAN (SSID)	222	Code	Unser Hotspot für die Tests. Das Passwort schreiben wir nicht auf.
MQTT-Broker	164.92.239.169:1883	Code, Node-RED	So vorgegeben.
Status-Topic	lindenpark/letterbox/noseryoung	Code	So steht es im Auftrag.
Alarm-Topic	lindenpark/letterbox/noseryoung/alarm	Node-RED, Code	Alarm und Status sind getrennt.
Messintervall	2 s	Code	Post wird schnell erkannt.
Bestätigung	3 Messungen (ca. 6 s)	Code	Einzelne Fehlmessungen lösen nichts aus.
Heartbeat	5 min	Code	Node-RED sieht, dass der ESP32 noch läuft.
Schwelle «Post»	Unter 5 cm oder verdeckt	Code	Leer misst der Sensor ca. 36 cm. Liegt Post darauf, kann er nicht messen.
Kontrollzeit	12:00, Mo–Fr	Node-RED	Bis Mittag sollte der Briefkasten geleert sein. Am Wochenende kommt keine Post.
Offline	Nach 30 min	Node-RED	Wenn 6 Heartbeats fehlen, ist der ESP32 wahrscheinlich ausgefallen.
Sirene	30 s	Code	Man hört sie gut, aber sie stört nicht zu lange.
Pins ESP32 Letterbox	Trig 5, Echo 18	Code	Sensor an 3,3 V, Echo direkt an GPIO 18.
Pins ESP32 Alarm	Lampe 2, Sirene 4, Knopf 13	Code	Freie Pins, die gut funktionieren.

Tabelle 6: Parameter der beiden Automatisierungen

4.2 Automatisierung 1: Posterkennung

Ziel: Der ESP32 erkennt, ob Post im Briefkasten liegt, und meldet den Status.

Ablauf:

1. Der Sensor misst alle 2 s die Distanz.
2. Unter 5 cm oder verdeckt bedeutet: Post ist da.
3. Nach 3 gleichen Messungen ändert sich der Status.
4. Der ESP32 sendet den Status an `lindenpark/letterbox/noseryoung`.
5. Node-RED speichert, wann Post gekommen ist und wann der Briefkasten geleert wurde.

```
// Mittelwert aus gültigen Messungen, -1 wenn keine gültig
float measureDistance() {
  const int SAMPLES = 5;
  float sum = 0;
  int valid = 0;
  for (int i = 0; i < SAMPLES; i++) {
    float d = measureOnce();
    if (d > 0) {
      sum += d;
      valid++;
    }
    delay(50);
  }
  return valid > 0 ? sum / valid : -1;
}

// ---------- Entscheidung ----------
// -1 bedeutet: Sensor verdeckt (oder defekt!) -> als "Post" gewertet
bool isMailDetected(float distance) {
  return distance < 0 || distance < MAIL_THRESHOLD_CM;
}

// Status nur wechseln, wenn STABLE_COUNT Messungen hintereinander gleich sind
void updateState(float distance) {
  bool raw = isMailDetected(distance);

  if (raw == lastRaw) {
    if (sameCount < STABLE_COUNT) sameCount++;
  } else {
    lastRaw = raw;
    sameCount = 1;
  }

  if (sameCount >= STABLE_COUNT && (!stateKnown || raw != mailState)) {
    mailState = raw;
    stateKnown = true;
    pendingReason = "change";
    publishPending = true;
    Serial.println(mailState ? ">> Status: POST VORHANDEN" : ">> Status: LEER");
  }
}
```

Abbildung 5: Messung und Schwelle im Code (ESP32 Letterbox)

```cpp
// ---------- MQTT senden ----------
bool publishStatus(const char *reason) {
  if (!client.isConnected()) return false;

  char buf[160];
  snprintf(buf, sizeof(buf),
           "{\"mail\":%s,\"distance\":%.1f,\"reason\":\"%s\",\"rssi\":%d,\"device\":\"%s\"}",
           mailState ? "true" : "false", lastDistance, reason,
           (int)WiFi.RSSI(), DEVICE_ID);

  // retain = true: Node-RED bekommt nach Neustart sofort den letzten Status
  bool ok = client.publish(PUB_TOPIC, std::string(buf), 0, true);
  Serial.print(ok ? "Gesendet: " : "Senden fehlgeschlagen: ");
  Serial.println(buf);
  return ok;
}

// ---------- MQTT-Callbacks (globale Funktionen, nicht static!) ----------
void onMqttConnect(esp_mqtt_client_handle_t c) {
  if (client.isMyTurn(c)) {
    Serial.println("MQTT verbunden");
    // nach (Neu-)Verbindung aktuellen Status nochmals senden
    if (stateKnown) {
      pendingReason = "reconnect";
      publishPending = true;
    }
  }
}

#if ESP_IDF_VERSION < ESP_IDF_VERSION_VAL(5, 0, 0)
esp_err_t handleMQTT(esp_mqtt_event_handle_t event) {
  client.onEventCallback(event);
  return ESP_OK;
}
#else
void handleMQTT(void *handler_args, esp_event_base_t base, int32_t event_id, void *event_data) {
  auto *event = static_cast<esp_mqtt_event_handle_t>(event_data);
  client.onEventCallback(event);
}
#endif

//            Arduino
```

Abbildung 6: Senden des Status per MQTT (ESP32 Letterbox)

```
// Topic: lindenpark/letterbox/<firma>
const firma = msg.topic.split('/')[2];
const p = msg.payload;

if (typeof p !== 'object' || typeof p.mail !== 'boolean') {
    node.warn('Ungültige Nachricht: ' + JSON.stringify(p));
    return null;
}

const key = 'box_' + firma;
const prev = flow.get(key) || {};
const now = Date.now();

const state = {
    mail: p.mail,
    distance: p.distance,
    lastSeen: now,
    mailSince: prev.mailSince || null,
    lastEmptied: prev.lastEmptied || null
};

// Post neu eingeworfen
if (p.mail && !prev.mail) state.mailSince = now;

// Wechsel Post -> leer = Briefkasten wurde geleert
if (!p.mail && prev.mail) {
    state.lastEmptied = now;
    state.mailSince = null;
}

flow.set(key, state);

const t = new Date(now).toLocaleTimeString('de-CH');
node.status({
    fill: p.mail ? 'yellow' : 'green',
    shape: 'dot',
    text: (p.mail ? 'Post vorhanden' : 'Leer') + ' · ' + t
});

msg.payload = Object.assign({ firma: firma }, state);
```

Abbildung 7: Node-RED – mqtt in und Function «Status / Leerung»

4.3 Automatisierung 2: Alarm

Ziel: Wenn der Briefkasten nicht geleert wurde, gibt es einen Alarm.

Ablauf:

1. Jeden Werktag um 12:00 startet Node-RED die Kontrolle.
2. Liegt Post im Briefkasten und wurde er heute nicht geleert, gibt es einen Alarm.
3. Node-RED zeigt ein Popup auf dem Dashboard und sendet den Alarm an …/alarm.
4. Der zweite ESP32 empfängt den Alarm. Die Sirene piept 30 s.
5. Mit dem Knopf kann man die Sirene früher ausschalten.
6. Die Lampe leuchtet, solange Post im Briefkasten liegt.

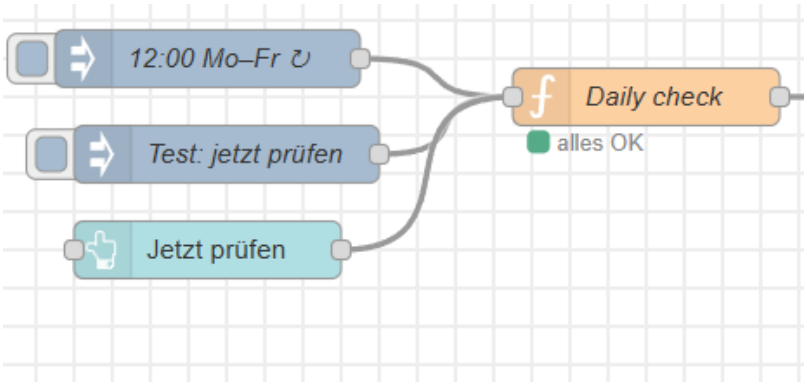


Abbildung 8: Node-RED-Flow für die tägliche Kontrolle



Abbildung 9: Function-Node «Daily check» in Node-RED

```
  // Alarm von Node-RED (oder Test-Button)
  client.subscribe(ALARM_TOPIC, [](const std::string &payload) {
    Serial.print("Alarm-Nachricht: ");
    Serial.println(payload.c_str());
    if (payload.find("\"alarm\":true") != std::string::npos)  alarmOnReceived = true;
    if (payload.find("\"alarm\":false") != std::string::npos) alarmOffReceived = true;
  });

  // Status vom Briefkasten -> steuert die LED
  client.subscribe(STATUS_TOPIC, [](const std::string &payload) {
    Serial.print("Status-Nachricht: ");
    Serial.println(payload.c_str());

    // Leerzeichen entfernen, damit auch {"mail": true} erkannt wird
    std::string p;
    for (char ch : payload) if (ch != ' ') p += ch;

    if (p.find("\"mail\":true") != std::string::npos) {
      mailPresent = true;
      Serial.println(">> Post vorhanden -> LED an");
    } else if (p.find("\"mail\":false") != std::string::npos) {
      mailPresent = false;
      alarmOffReceived = true;   // geleert -> auch Sirene aus
      Serial.println(">> Leer -> LED aus");
    }
  });
}
```

Abbildung 10: ESP32 Alarm – Subscribe auf Alarm- und Status-Topic

```
void loop() {
  if (WiFi.status() != WL_CONNECTED) setup_wifi();

  bool blink = (millis() / BLINK_MS) % 2 == 0;

  // 1) Knopf -> Sirene aus
  if (buttonPressed()) {
    Serial.println("Knopf gedrückt");
    stopAlarm("Knopf");
  }

  // 2) Sirene ein
  if (alarmOnReceived) {
    alarmOnReceived = false;
    alarmActive = true;
    alarmStart = millis();
    Serial.println(">> SIRENE EIN (30 s)");
  }

  // 3) Sirene aus per MQTT / geleert
  if (alarmOffReceived) {
    alarmOffReceived = false;
    stopAlarm("MQTT / geleert");
  }

  // 4) Nach 30 s automatisch aus
  if (alarmActive && millis() - alarmStart >= ALARM_DURATION_MS) {
    stopAlarm("30 s vorbei");
  }

  // 5) Ausgänge
  setLed(mailPresent);              // LED-Modul blinkt selbst
  setSiren(alarmActive && blink);   // Sirene piept nur bei Alarm

  delay(10);
}
```

Abbildung 11: ESP32 Alarm – loop() mit Knopf, Sirene und Lampe

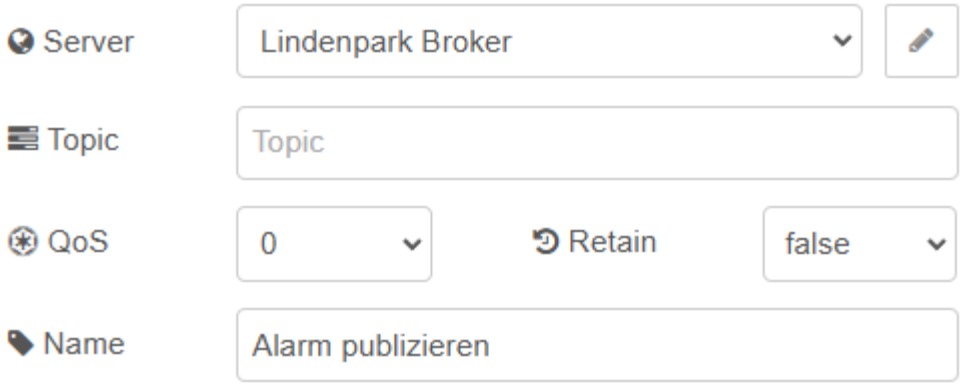


Abbildung 12: Node-RED – mqtt out Node für …/alarm

```
// Prüft alle bekannten Briefkasten
const keys = flow.keys().filter(k => k.startsWith('box_'));
if (keys.length === 0) {
    node.warn('Noch keine Daten von einem Briefkasten');
    return null;
}

const startOfDay = new Date();
startOfDay.setHours(0, 0, 0, 0);

const out = [];
let alarms = 0;

for (const k of keys) {
    const s = flow.get(k);
    const firma = k.slice(4);
    const emptiedToday = s.lastEmptied && s.lastEmptied >= startOfDay
    const alarm = s.mail === true && !emptiedToday;
    if (alarm) alarms++;

    out.push({
        topic: 'lindenpark/letterbox/' + firma + '/alarm',
        payload: {
            alarm: alarm,
            firma: firma,
            mail: s.mail,
            mailSince: s.mailSince,
            lastEmptied: s.lastEmptied,
            text: alarm ? 'Briefkasten ' + firma + ' wurde heute nich
        }
    });
}

node.status({
    fill: alarms ? 'red' : 'green', shape: 'dot',
    text: alarms ? alarms + ' Alarm(e)' : 'alles OK'
});

return [out];
```

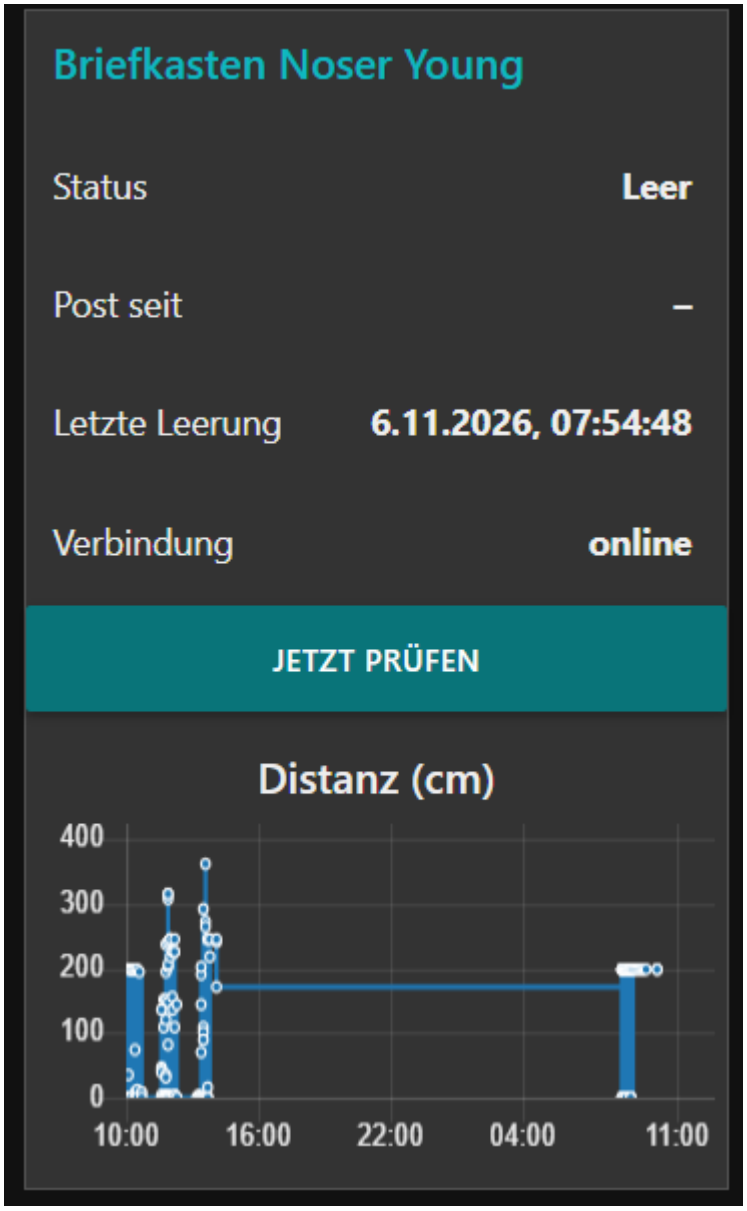


Abbildung 13: Dashboard

4.4 Abgleich mit den Anforderungen

Anforderung	Umsetzung
Erkennen, ob Post da ist	Ultraschallsensor am Boden
Status regelmässig senden	Bei jeder Änderung und alle 5 min (Heartbeat)
Topic [standort]/letterbox/[firma]	lindenpark/letterbox/noseryoung
Täglich melden, wenn nicht geleert	Kontrolle um 12:00, Alarm per MQTT, Dashboard und Sirene
Optional: sichtbar machen	Dashboard und Lampe

Tabelle 7: Abgleich mit den Anforderungen aus dem Auftrag

5 Einbindung der Geräte

Autor/in: Cristiano Grgic

5.1 Verdrahtung

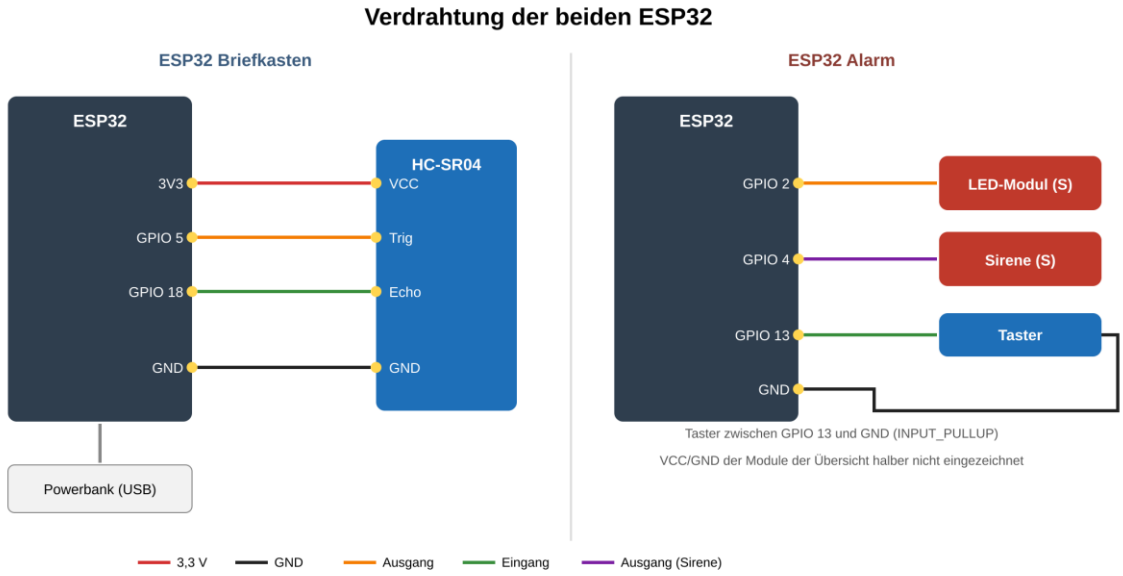


Abbildung 14: Verdrahtung des ESP32 Briefkasten und des ESP32 Alarm

ESP32 Briefkasten

- **HC-SR04:** VCC an 3,3 V, GND an GND, Trig an GPIO 5, Echo direkt an GPIO 18.
- **Speisung:** Powerbank am USB-Anschluss des ESP32.

ESP32 Alarm

- **LED-Modul:** Signal an GPIO 2.
- **Sirene / Buzzer:** Signal an GPIO 4.
- **Taster:** zwischen GPIO 13 und GND. Der ESP32 nutzt den internen Pull-up (INPUT_PULLUP), gedrückt = LOW.

5.2 Fotos der Geräte

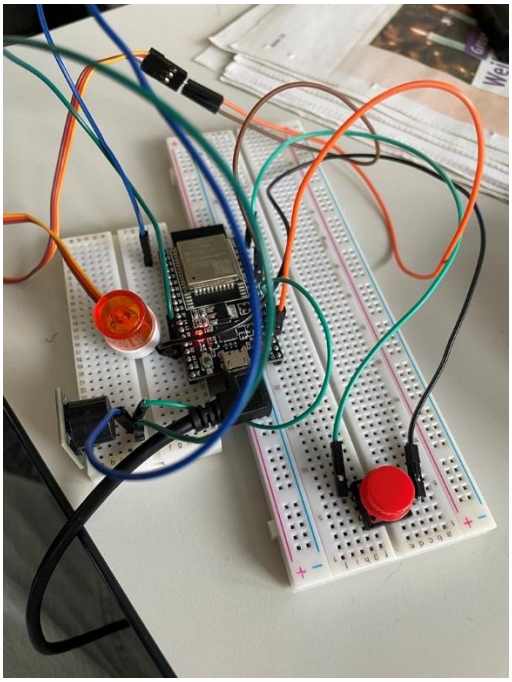


Abbildung 15: Das Alarmsystem (ESP32 Alarm)

Komponenten des Alarmsystems

- Knopf
- LED-Licht
- ESP32
- Jumper-Kabel (male-male)
- Piezo-Buzzer-Modul
- Kabel USB-A auf Micro-USB

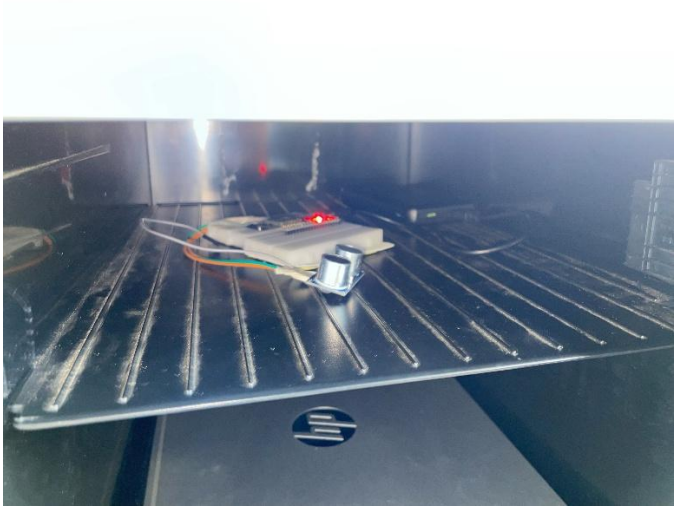


Abbildung 16: Das Erkennungssystem im Briefkasten (ESP32 Briefkasten)

Komponenten des Erkennungssystems

- Powerbank

- Ultraschallsensor HC-SR04
- Jumper-Kabel (male-female)
- Kabel USB-A auf Micro-USB
- ESP32
- Steckplatine

5.3 Erfahrungen beim Aufbau

Tests mit den Sensoren

Beim ersten Test stellten wir fest, dass mehrere Sensoren sinnvoll wären, um eine höhere Genauigkeit zu erreichen. Mit nur einem Sensor könnte ein Brief neben dem Sensor landen und somit nicht erkannt werden. Deshalb entschieden wir uns, drei Sensoren zu verwenden. Sobald einer der Sensoren etwas erkennt, wird eine Nachricht übermittelt.

Dabei hatten wir zunächst Schwierigkeiten, die drei Sensoren gleichzeitig anzuschliessen, da wir bisher nur mit einzelnen Sensoren gearbeitet hatten. Schlussendlich konnten wir das Problem lösen, indem wir jeden Sensor an einen eigenen Pin des ESP32 anschlossen.

Bei den ersten Praxistests draussen bemerkten wir, dass die Distanzmessung sehr ungenau war. Unsere Vermutung war, dass im Briefkasten Echos entstehen. Die Ultraschallwellen werden dabei mehrfach im Inneren reflektiert, anstatt direkt zum Sensor zurückzukehren. Deshalb überlegten wir, weniger Sensoren zu verwenden oder die Sensoren senkrecht nach oben auszurichten, damit die Signale direkt zurückgeworfen werden. Ideal wäre zudem eine harte, flache Oberfläche als Reflexionsfläche.

Verbesserungen

Bei einem weiteren Test mit einer flachen Oberfläche funktionierte die Messung deutlich besser, und die Sensoren arbeiteten zuverlässig. Später stellten wir jedoch fest, dass ein eingeworfener Brief nicht immer erkannt wurde. Deshalb mussten wir den Distanzwert anpassen, ab dem der Sensor auslöst.

Die Verbindung und Datenübertragung haben wir mit Node-RED umgesetzt.

Für das Endergebnis benötigten wir noch eine Batterie. Cristiano brachte eine Powerbank mit, damit wir testen konnten, ob die Stromversorgung auf diese Weise zuverlässig funktioniert.

Lampe und Alarm

Am zweiten Tag haben wir versucht, die Lampen mit dem Internet zu verbinden, sodass die Lampen angehen, wenn die Sensoren etwas erkennen. Das haben wir mithilfe von Node-RED gemacht.

Danach mussten wir am Alarm weiterarbeiten. Der Alarm klingelte, aber man konnte ihn nicht einfach ausschalten. Man musste manuell in die Arduino IDE gehen und dort den Code ändern, was sehr aufwendig ist. Die zweite Variante wäre, den Alarm auszustecken. Das ist weniger aufwendig, aber trotzdem nervig. Also haben wir uns entschieden, die Alarme mithilfe von Node-RED zu

bestimmten Zeiten einzuschalten, falls man vergisst, die Post zu nehmen, und sie mit einem einfachen Knopf auszuschalten.

Endstand

Am Ende verwenden wir nur einen HC-SR04. Er ist am Boden des Briefkastens montiert und misst nach oben. Im leeren Briefkasten misst er ca. 36 cm. Liegt Post darauf, ist der Abstand unter 5 cm oder der Sensor ist verdeckt.

6 Erweiterungen und Weiterführung

Autor/in: Yurii Myronenko

In diesem Kapitel zeigen wir, wie man das Projekt erweitern kann. Danach beschreiben wir, an welche Regeln wir uns beim Programmieren gehalten haben. Zum Schluss listen wir auf, was ein anderes Team braucht, um das Projekt weiterzuführen.

6.1 Mögliche Erweiterung: weitere Firmen der Noser Gruppe

Beschreibung

Im Lindenpark gibt es weitere Firmen der Noser Gruppe: AKROS, Bucher+Suter, DANEXiS, FROX und Noser Engineering. Jede Firma bekommt einen eigenen smarten Briefkasten. Alle Briefkästen senden an denselben Broker, aber jede Firma hat ihr eigenes Topic, zum Beispiel lindenpark/letterbox/akros.

Unser System ist dafür schon vorbereitet. Node-RED hört auf `lindenpark/letterbox/+` und empfängt damit alle Firmen. Die Function «Daily check» prüft automatisch alle gespeicherten Briefkästen (`box_…`).

Zusätzliche Komponenten (pro Briefkasten)

Komponente	Anzahl	Wozu
ESP32	1	Messen und Status senden
HC-SR04	1	Post erkennen
Powerbank	1	Strom im Briefkasten
Kabel	9	Verbindung
ESP32 + Sirene + Lampe + Knopf (optional)	1 Set	Alarm direkt bei der Firma

Tabelle 8: Zusätzliche Komponenten pro Briefkasten

WLAN: Für mehrere Briefkästen ist ein Handy-Hotspot nicht stabil genug. Besser ist ein gemeinsames WLAN im Gebäude, mit dem alle Briefkästen verbunden sind.

Anpassungen

Wo	Was ändern
Code ESP32 Letterbox	Nur `PUB_TOPIC` (z. B. `lindenpark/letterbox/akros`) und `DEVICE_ID` (z. B. `ESP-32-Post-akros`). Die ID muss eindeutig sein.
Code ESP32 Alarm	Nur `ALARM_TOPIC` und `STATUS_TOPIC` auf die neue Firma ändern.
Node-RED Logik	Nichts. Neue Firmen werden automatisch erkannt.
Node-RED Dashboard	Pro Firma eine eigene Gruppe mit Status und letzter Leerung.

Tabelle 9: Nötige Anpassungen für eine neue Firma

6.2 Codequalität und Coding Conventions

Wir haben uns an folgende Regeln gehalten:

- **Konstanten in GROSSBUCHSTABEN**, zum Beispiel `ALARM_DURATION_MS`, `BLINK_MS`, `ALARM_TOPIC`. Alle Einstellungen stehen oben im Code. So findet man sie schnell.
- **Funktionen in camelCase** mit klaren Namen, zum Beispiel `stopAlarm()`, `buttonPressed()`, `setLed()`.
- **Code in Blöcke aufgeteilt**, mit Kommentaren wie `// ---------- WLAN ----------` oder `// 1) Knopf -> Sirene aus`.
- `millis()` **statt langer** `delay()`: Der ESP32 bleibt nicht stehen. Er kann gleichzeitig messen, den Knopf prüfen und MQTT-Nachrichten empfangen.
- **Einheiten im Namen**, zum Beispiel `_MS` für Millisekunden und `_CM` für Zentimeter. So gibt es keine Verwechslungen.
- **Keine Passwörter im Git**: Im Git-Repository steht statt des echten WLAN-Passworts nur ein Platzhalter.

6.3 Unterlagen zum Fortführen der Arbeit

Ein anderes Team findet alles in unserem Git-Repository: https://github.com/Noe-NoserYoung/uek216.git

Struktur

Ordner / Datei	Inhalt
`/firmware/letterbox`	Code ESP32 Briefkasten
`/firmware/letterbox_alarm`	Code ESP32 Alarm
`/nodered/flows.json`	Export des Node-RED-Flows
`/docs`	Dokumentation, Schaltpläne
`README.md`	Anleitung

Tabelle 10: Struktur des Git-Repositorys

Inhalt des README

- Benötigte Bibliothek: ESP32MQTTClient, Version 1.1.5
- WLAN und Broker im Code einstellen (Platzhalter durch eigene Zugangsdaten ersetzen)
- Code mit der Arduino IDE hochladen
- Flow in Node-RED importieren (Menü → Import → `flows.json`)
- Dashboard öffnen: `http://164.92.239.169:1884/ui`

Bekannte Probleme

- Nach einem Neustart von Node-RED ist die Zeit der letzten Leerung weg. Lösung: Persistent Context in Node-RED aktivieren.
- Der Sensor zeigt «verdeckt» auch, wenn ein Kabel locker ist. Das kann man nicht vom Fall «Post» unterscheiden.
- Mit einem Handy-Hotspot ist die Verbindung manchmal instabil.

- Node-RED nutzt die Uhrzeit des Servers. Die Kontrollzeit muss eventuell angepasst werden.

Nächste Schritte

- Akku mit Deep Sleep statt Powerbank
- Benachrichtigung per Teams oder E-Mail
- Gewichtssensor (HX711) für eine sicherere Erkennung

7 Zugriffskonzept für Geräte und Daten

Autor/in: Yurii Myronenko

In diesem Abschnitt zeigen wir, wer auf welche Teile des Systems zugreifen kann und welche Rechte diese Personen oder Geräte haben. So sehen wir, wo das System gut geschützt ist und wo es noch Lücken gibt.

Wer	Zugriff auf	Rechte	Wie geschützt
Lernende Noser Young	Dashboard (http://164.92.239.169:1884/ui)	Status lesen, Button «Jetzt prüfen»	Benutzername und Passwort (ohne HTTPS)
Projektteam	Node-RED-Editor	Flow lesen und ändern	Benutzername und Passwort (ohne HTTPS)
Projektteam	ESP32 Briefkasten und ESP32 Alarm	Code hochladen	USB (physischer Zugang) oder OTA mit Passwort
Projektteam	Git-Repository	lesen und ändern	GitHub-Konto; WLAN-Passwort nur als Platzhalter
Berufsbildner	Git-Repository	lesen und ändern	GitHub-Konto
Betreiber des Servers (Kursleitung)	Server 164.92.239.169 mit Broker und Node-RED	Administration	Nur die Kursleitung hat Zugriff auf den Server selbst. Wir sehen nur Node-RED und kommen dort nur mit Login hinein.
Alle im Netzwerk	MQTT-Broker, Topics lindenpark/letterbox/#	lesen und senden	kein Schutz (geplant: Anmeldung und ACL, siehe Kapitel 8)

Tabelle 11: Zugriffskonzept

8 Schutzkonzept für die Geräte

Autor/in: Yurii Myronenko

8.1 Ausgangslage / Risiken

Node-RED ist teilweise geschützt: Für den Editor und das Dashboard braucht man einen Benutzernamen und ein Passwort. Die Verbindung läuft aber über HTTP ohne TLS. Der MQTT-Broker ist dagegen nicht geschützt. Der ESP32 verbindet sich über Port 1883 ohne TLS, die Daten werden also unverschlüsselt übertragen. Für die Verbindung zum Broker braucht man keine Anmeldung. Das WLAN-Passwort und die Adresse des Brokers stehen direkt im Quellcode. Im Git-Repository haben wir das Passwort durch einen Platzhalter ersetzt.

Das bedeutet: Eine fremde Person kann lindenpark/letterbox/# abonnieren und sehen, ob Post im Briefkasten liegt. Sie kann auch selbst {"alarm":true} senden und so die Sirene einschalten, oder mit {"mail":false} eine Leerung vortäuschen. Zusätzlich gibt es technische Risiken: Der ESP32 kann ausfallen, oder es kommen fehlerhafte Nachrichten an.

8.2 Massnahme 1: Offline-Erkennung

Risiko

Der ESP32 fällt aus, zum Beispiel weil die Powerbank leer ist oder das WLAN nicht funktioniert. Das Dashboard zeigt dann weiter den letzten Status, zum Beispiel «Leer». Niemand merkt, dass das System nicht mehr funktioniert. Post wird nicht mehr erkannt, und es gibt keinen Alarm.

Massnahme

Der ESP32 sendet regelmässig ein Lebenszeichen (Heartbeat). Node-RED prüft, ob Nachrichten ankommen. Wenn zu lange nichts kommt, zeigt das Dashboard «OFFLINE».

Umsetzung

1. Der ESP32 sendet alle 5 Minuten einen Heartbeat, auch wenn sich der Status nicht ändert (HEARTBEAT_MS im Code).
2. In Node-RED startet jede Nachricht einen Trigger-Node neu. Er wartet 30 Minuten. Kommt in dieser Zeit keine Nachricht, sendet er «OFFLINE» an das Dashboard.
3. 30 Minuten entsprechen 6 verpassten Heartbeats. So gibt es keinen Fehlalarm bei einer kurzen Störung.
4. Kommt wieder eine Nachricht, zeigt das Dashboard automatisch wieder «online».

```
// 2) Heartbeat, damit Node-RED weiss, dass das Gerät lebt
if (stateKnown && !publishPending && millis() - lastPublish >= HEARTBEAT_MS) {
  pendingReason = "heartbeat";
  publishPending = true;
}

// 3) Senden (bleibt "pending", bis es wirklich geklappt hat)
if (publishPending && stateKnown) {
  if (publishStatus(pendingReason)) {
    publishPending = false;
    lastPublish = millis();
  }
}

delay(10);
```

Abbildung 17: Heartbeat im Code (ESP32 Letterbox)

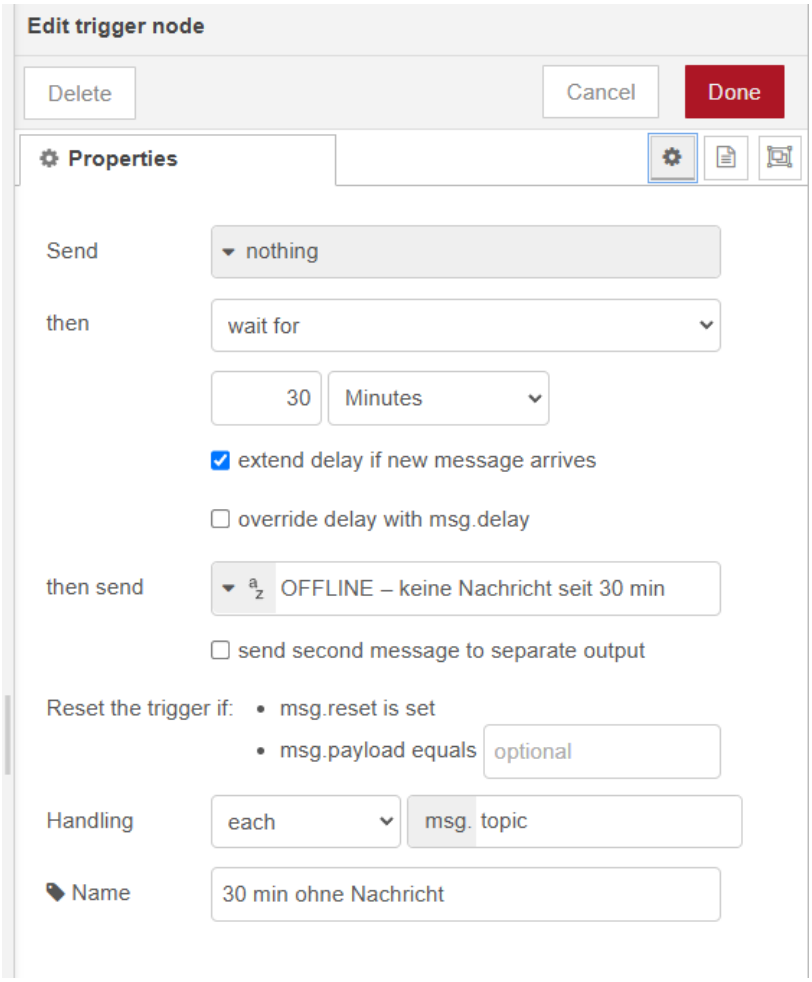


Abbildung 18: Einstellungen des Trigger-Nodes «30 min ohne Nachricht»

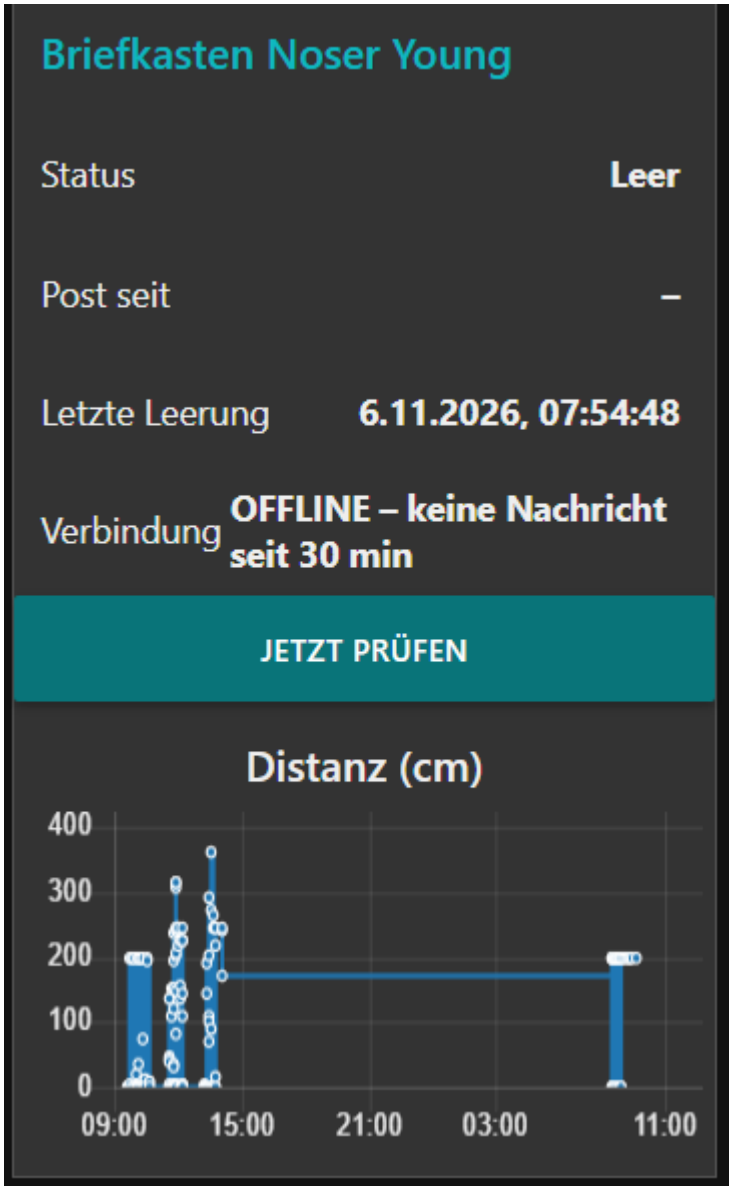


Abbildung 19: Dashboard mit Anzeige «OFFLINE»

Test: Für den Test haben wir die Zeit im Trigger-Node auf 1 Minute gesetzt und den ESP32 ausgeschaltet. Nach 1 Minute erschien «OFFLINE». Nach dem Einschalten erschien wieder «online».

8.3 Massnahme 2: Prüfung der Nachrichten

Risiko
Auf dem Topic kann eine fehlerhafte oder manipulierte Nachricht ankommen, zum Beispiel einfacher Text statt JSON oder eine Nachricht ohne das Feld mail. Ohne Prüfung würde Node-RED einen falschen Status speichern. Dann wären auch die Zeit der letzten Leerung und die tägliche Kontrolle falsch.
Massnahme
Node-RED prüft jede Nachricht, bevor sie verarbeitet wird. Nachrichten im falschen Format werden verworfen und nicht gespeichert.
Umsetzung

1. Der mqtt-in-Node ist auf «parsed JSON object» eingestellt. Nachrichten, die kein gültiges JSON sind, werden nicht als Objekt weitergegeben.

2. Am Anfang der Function «Status / Leerung» wird geprüft, ob die Nachricht ein Objekt ist und ob mail true oder false ist:

```
if (typeof p !== 'object' || typeof p.mail !== 'boolean') {
    node.warn('Ungültige Nachricht: ' + JSON.stringify(p));
    return null;
}
```

3. Ist die Nachricht ungültig, erscheint eine Warnung im Debug-Fenster, und der gespeicherte Status bleibt unverändert.

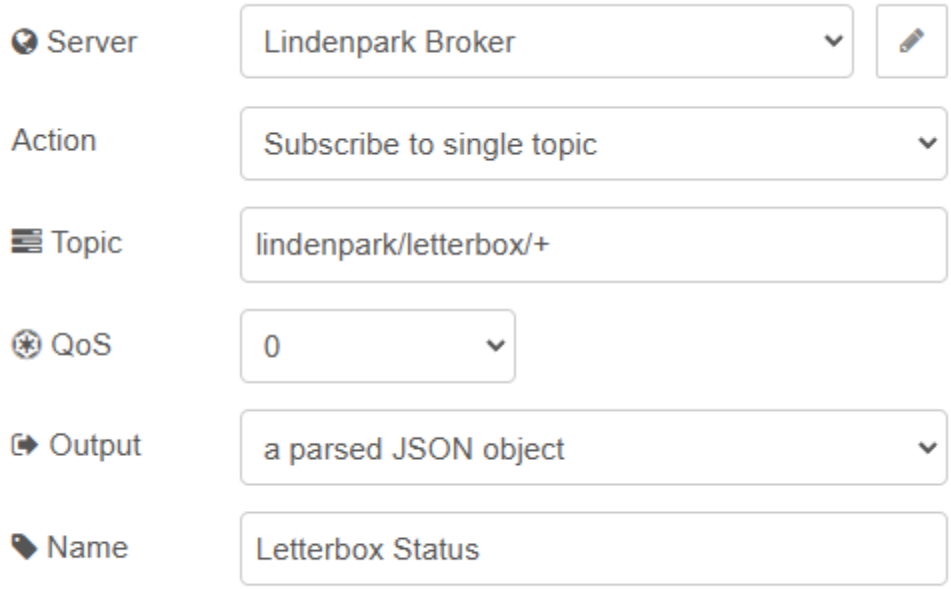


Abbildung 20: mqtt-in-Node mit Ausgabe «a parsed JSON object»



Abbildung 21: Function «Status / Leerung» mit Prüfung am Anfang

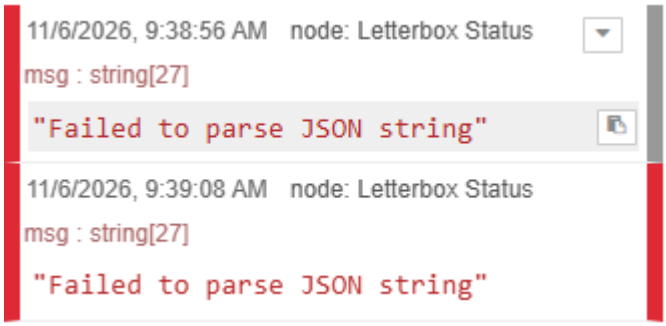


Abbildung 22: Debug-Fenster bei einer ungültigen Nachricht

Der mqtt-in-Node meldet «Failed to parse JSON string». Die Nachricht wird nicht weiterverarbeitet.

Test: Mit MQTT Explorer haben wir hallo und {"test":1} an lindenpark/letterbox/noseryoung gesendet. Beide Nachrichten wurden verworfen, und der Status auf dem Dashboard blieb gleich.

Grenze: Eine Nachricht im richtigen Format, zum Beispiel {"mail":false}, wird trotzdem angenommen. Dagegen hilft nur eine Anmeldung am Broker (siehe Tabelle 12, «geplant»).

8.4 Übersicht

Risiko	Massnahme	Umsetzung im Projekt	Status
ESP32 fällt aus, niemand merkt es	Offline-Erkennung	Heartbeat alle 5 min, Trigger 30 min, Anzeige auf dem Dashboard	umgesetzt
Fehlerhafte Nachrichten zerstören den Status	Prüfung der Nachrichten	JSON-Parser und Prüfung von mail in «Status / Leerung»	umgesetzt
Daten können abgehört werden	TLS-Verschlüsselung	Broker auf Port 8883 mit TLS, ESP32 mit mqtts://	geplant
Jeder kann Nachrichten senden (z. B. falscher Alarm)	Anmeldung am Broker	Benutzername und Passwort, Rechte pro Topic (ACL)	geplant
WLAN-Passwort ist im Git sichtbar	Zugangsdaten aus dem Code entfernen	Im Git nur ein Platzhalter statt dem Passwort. Geplant: secrets.h und .gitignore	teilweise umgesetzt
Unbefugter Zugriff auf Node-RED (Flow ändern, Dashboard sehen)	Anmeldung und TLS für Node-RED	Benutzername und Passwort für Editor und Dashboard. HTTPS fehlt noch.	teilweise umgesetzt

Tabelle 12: Übersicht der Schutzmassnahmen

9 Konzept für regelmässige Updates

Autor/in: Noé Bouvard

9.1 Vorgehen beim Aktualisieren

Firmware ESP32

- Code im Git-Repository ändern.
- In der Arduino IDE kompilieren.
- Über OTA (Netzwerk-Port in der Arduino IDE, Passwort) oder per USB hochladen. Der OTA-Code muss im neuen Code enthalten sein.
- Im Serial Monitor oder auf dem Dashboard prüfen, ob der ESP32 wieder läuft.

Node-RED-Flow

- flows.json im Git-Repository aktualisieren.
- In Node-RED importieren (Menü → Import → flows.json) und «Deploy» drücken.

Bibliotheken und Board-Paket

- In der Arduino IDE über den Library Manager bzw. Boards Manager aktualisieren und danach den Code neu testen.

9.2 Quellen der Updates

Was	Quelle
Eigener Code und Node-RED-Flow	Git-Repository https://github.com/Noe-NoserYoung/uek216.git
Bibliothek ESP32MQTTClient	Arduino Library Manager
Board-Paket arduino-esp32 (inkl. ArduinoOTA)	Espressif, über den Boards Manager der Arduino IDE (GitHub espressif/arduino-esp32)
Node-RED und node-red-dashboard	Node-RED-Projekt (nodered.org), Palette Manager in Node-RED

Tabelle 13: Quellen der Updates

9.3 OTA-Update (Over the Air)

Wir haben auf unserem ESP32 OTA-Updates (Over the Air) aktiviert. Dafür sind wir dem Tutorial von SunFounder gefolgt (siehe Quellenverzeichnis).

Zuerst mussten wir in der Arduino IDE unter File > Examples > ArduinoOTA das Beispiel BasicOTA öffnen und auf den ESP32 laden. Danach mussten wir nur noch unseren Code mit dem OTA-Code (bereitgestellt durch Claude) verknüpfen und ein Passwort eingeben. Ab jetzt muss der OTA-Code bei jedem Upload im aktualisierten Code enthalten sein.

Momentan hat nur der ESP32 Alarm OTA.

```
// ---------- OTA ----------
const char *OTA_HOSTNAME = "letterbox-alarm";
const char *OTA_PASSWORD = "OTA PASWORT";    // eigenes Passwort wählen
```

Abbildung 23: OTA-Einstellungen im Code (ESP32 Alarm)

```
// ---------- OTA ----------
void setup_ota() {
  ArduinoOTA.setHostname(OTA_HOSTNAME);
  ArduinoOTA.setPassword(OTA_PASSWORD);

  ArduinoOTA.onStart([]() {
    // Sirene aus, sonst bleibt sie während des Updates hängen
    alarmActive = false;
    setSiren(false);
    Serial.println("OTA Start");
  });
  ArduinoOTA.onEnd([]() { Serial.println("\nOTA Ende"); });
  ArduinoOTA.onProgress([](unsigned int p, unsigned int t) {
    Serial.printf("OTA: %u%%\r", p * 100 / t);
  });
  ArduinoOTA.onError([](ota_error_t e) { Serial.printf("OTA Fehler [%u]\n", e); });

  ArduinoOTA.begin();
}
```

Abbildung 24: Funktion setup_ota() im Code (ESP32 Alarm)

10 Testfälle

Autor/in: Cristiano Grgic

10.1 Testprotokoll

Nr.	Testfall	Vorgehen	Erwartetes Resultat	Resultat	OK / NOK
T1	Offline-Erkennung	Zeit im Trigger-Node auf 1 Minute gesetzt, ESP32 ausgeschaltet, danach wieder eingeschaltet	Dashboard zeigt «OFFLINE», danach wieder «online»	Nach 1 Minute «OFFLINE», nach dem Einschalten «online»	OK
T2	Ungültige Nachrichten	Mit MQTT Explorer «hallo» und {"test":1} an lindenpark/letterbox/noseryoung gesendet	Nachrichten werden verworfen, Status bleibt gleich	Beide verworfen, Status unverändert	OK
T3	Post wird erkannt	Brief in den leeren Briefkasten auf den Sensor legen	Nach ca. 6 s: Dashboard «Post vorhanden», «Post seit» mit Uhrzeit, LED leuchtet	Lief wie erwartet	OK
T4	Leerung wird erkannt	Post aus dem Briefkasten nehmen	Nach ca. 6 s: Dashboard «Leer», «Letzte Leerung» mit Uhrzeit, LED aus	Leerung wird erkannt	OK
T5	Täglicher Alarm	Post liegt, heute nicht geleert. Button «Jetzt prüfen» drücken (statt 12:00 warten)	Popup auf dem Dashboard, Nachricht auf …/noseryoung/alarm , Sirene piept	Der Alarm funktioniert wie geplant	OK
T6	Sirene mit Knopf aus	Alarm auslösen (T5), dann Knopf drücken	Sirene geht sofort aus	Das hat funktioniert	OK
T7	Sirene nach 30 s aus	Alarm auslösen (T5), Knopf nicht drücken	Sirene geht nach 30 s automatisch aus	Lief wie erwartet	OK
T8	Kein Alarm nach Leerung	Post einwerfen und wieder herausnehmen, dann «Jetzt prüfen»	Kein Alarm, Status «alles OK»	Lief wie erwartet	OK
T9	Reconnect	Hotspot aus- und nach 1 min wieder einschalten	ESP32 verbindet sich neu, sendet «reconnect», Dashboard wieder «online»	Lief wie erwartet	OK

Nr.	Testfall	Vorgehen	Erwartetes Resultat	Resultat	OK / NOK
T10	OTA-Update	Geänderten Code über den Netzwerk-Port hochladen (ohne USB)	Upload erfolgreich, ESP32 startet mit neuem Code	Das Over the Air Update lief wie erwartet	OK

Tabelle 14: Testprotokoll

Datum der Tests:

- 04.11.2026: T1-T4
- 05.11.2026: T5-T8
- 06.11.2026: T9-T10

Getestet von: Noé Bouvard, Cristiano Grgic, Yurii Myronenko

Quellenverzeichnis

- Noser Young AG: LB-Projektauftrag «Letterbox», Modul 216 – Internet of Everything.
- Noser Young AG: Leistungsbeurteilung Projektarbeit – Bewertungsraster, 29.10.2026.
- SunFounder: ESP32 OTA Updates – A Complete Guide to ArduinoOTA and ElegantOTA Firmware Upgrades. https://www.sunfounder.com/blogs/news/esp32-ota-updates-a-complete-guide-to-arduinoota-and-elegantota-firmware-upgrades
- Espressif: arduino-esp32, Beispiel libraries/ArduinoOTA/examples/BasicOTA/BasicOTA.ino und Header ArduinoOTA.h (GitHub).
- Espressif: Dokumentation ArduinoOTA, docs.espressif.com.
- Bibliothek ESP32MQTTClient, Version 1.1.5 (Arduino Library Manager).

Eingesetzte KI-Hilfsmittel

- Claude (Anthropic): Informationen zur Verkabelung, OTA-Beispielcode, Grafiken «Zusammenspiel der Komponenten» und «Verdrahtung», Korrektur von Rechtschreibung und Grammatik, Struktur der Dokumentation sowie Tabellen und Stichpunkte aus unserem Code (Kapitel 2, 5, 7, 9, 10).